

Exploring distributed solutions using PySpark for Financial Fraud Transaction Detections

Mohammed Adnan*, Mohammed Tahir Shaik*, Fahad Al Mannan*

*School of Computer Science, University of Nottingham, Nottingham, UK

Emails: psxm4@nottingham.ac.uk, psxms35@nottingham.ac.uk, psxfl12@nottingham.ac.uk

Abstract—Financial fraud detection is an extreme class imbalance problem, with fraud representing only about 0.13% of the 6.3 million PaySim transactions [1] PaySim Kaggle dataset. This study evaluates distributed imbalance handling techniques in PySpark, including undersampling, SMOTE oversampling, and class weighted learning, paired with a custom histogram based Decision Tree, Gradient-Boosted Trees, and AdaBoost. The implementations are compared in terms of fraud-detection performance, size-up behaviour, and sensitivity to partition count. Results show that imbalance handling strongly affects the precision-recall trade-off, SMOTE and undersampling improve fraud recall but can increase false positives, while class-weighted AdaBoost achieves strong recall and precision after threshold tuning. Scalability results show sub-linear behaviour for the histogram Decision Tree and undersampled GBT pipelines due to bounded summaries and Spark scheduling overhead, while AdaBoost scales near linearly with boosting rounds. Partition count experiments show mostly stable accuracy, suggesting near partition invariant classification quality.

Index Terms—Fraud detection, class imbalance, PySpark, distributed machine learning, undersampling, SMOTE, class weights, Decision Tree, Gradient Boosted Trees, AdaBoost, scalability, partitioning.

I. INTRODUCTION

Fraud detection in large scale financial transaction data is challenging because fraudulent cases are extremely rare. In the PaySim dataset, fraud accounts for only about 0.13% of more than 6.3 million transactions. A classifier trained directly on this distribution can achieve high overall accuracy while missing most fraud cases, so accuracy alone is not sufficient. Other Metrics are more meaningful for evaluating model usefulness.

This report investigates distributed class-imbalance handling in PySpark. Three approaches are considered, undersampling, SMOTE-based oversampling, and class-weighted learning. These are paired with three tree-based classifiers: a custom global histogram-based Decision Tree, PySpark MLlib Gradient-Boosted Trees, and a global AdaBoost ensemble using weighted decision-tree weak learners. The aim is to compare both fraud-detection performance and distributed execution behaviour.

Local undersampling and partition-local SMOTE require little or no network transfer, while global undersampling, shuffled SMOTE, histogram aggregation, and boosted-tree training introduce shuffles or repeated full-data passes.

Scalability is also treated as a central concern. Size-up experiments examine whether runtime grows linearly, sub-

linearly, or super-linearly as data size increases. Partition-count experiments examine whether fraud-detection quality remains stable as PySpark parallelism changes. Ideally, accuracy should remain partition-invariant because the same logical training data should produce the same model regardless of how rows are divided.

This report investigates the following research questions:

RQ1: How can undersampling, SMOTE oversampling, and class-weighted learning be implemented in PySpark for imbalanced fraud classification using Decision Tree, Gradient-Boosted Tree, and AdaBoost classifiers?

RQ2: How do these distributed implementations scale as training data size increases?

RQ3: How does changing the number of PySpark partitions affect fraud-detection accuracy?

II. RELATED WORK

Class imbalance is a major challenge in fraud detection because fraudulent transactions are rare compared with legitimate activity. Chawla et al. [2] introduced SMOTE, which generates synthetic minority-class examples by interpolating between existing minority instances and their nearest neighbours. This is important for fraud detection because accuracy alone can be misleading on highly imbalanced data, and minority-class recall often requires explicit imbalance handling.

Freund and Schapire [3] introduced AdaBoost, where weak learners are trained sequentially, misclassified examples receive higher weights, and each learner is combined using an alpha value based on weighted error. This provides the basis for the class-weighted AdaBoost implementation in this report, where row weights are updated using distributed PySpark DataFrame operations.

Tree-based models are widely used for imbalanced classification because they capture non-linear relationships in transaction data. The custom Decision Tree in this work is motivated by LightGBM's histogram-based split finding [4], which reduces split-search cost by using binned feature summaries. Gradient-Boosted Trees and AdaBoost are also relevant because boosting methods repeatedly focus on difficult or previously misclassified examples, making them suitable for imbalanced learning [5].

In distributed PySpark, imbalance techniques must also be judged by data movement. Local sampling and partition-level SMOTE can avoid shuffles, while global sampling, shuffled SMOTE, histogram aggregation, and tree training require

wider transformations and higher communication cost [6]. Prior distributed oversampling methods such as SMOTE-BD [7] and Approx-SMOTE [8] show that SMOTE can be scaled, but nearest-neighbour search and minority-data movement remain important bottlenecks.

Overall, prior work suggests that no single imbalance method is universally best.

III. METHODOLOGY

This project evaluates three PySpark implementations for class-imbalance fraud detection on the PaySim dataset. The methods are organised according to the three individual implementations: class-weighted AdaBoost, distributed SMOTE with a custom Decision Tree, and sampling strategies with Gradient-Boosted Trees.

A. Class-Weighted Global AdaBoost

The AdaBoost implementation uses the full PaySim dataset and handles imbalance through class-balanced row weights. The raw CSV is loaded into a PySpark DataFrame, with transaction step, type, amount, origin and destination balances retained. The fraud label is cast to the binary `label` column. Feature engineering adds origin and destination balance deltas, zero-balance indicators, absolute origin balance error, and a binary balance-inconsistency flag. Transaction type is indexed and one-hot encoded, producing a 17-dimensional feature vector.

The data is split into train, validation and test sets using a fixed seed. The preprocessing pipeline is fitted only on the training data to avoid leakage. The prepared training set contains approximately 4.45 million rows, including 5,809 fraud cases, and is repartitioned before training. Initial weights are class-balanced so that fraud and non-fraud examples each contribute half of the total weight:

$$w_i = \begin{cases} \frac{1}{2N_1}, & y_i = 1 \\ \frac{1}{2N_0}, & y_i = 0 \end{cases} \quad (1)$$

Each boosting round trains a PySpark decision-tree weak learner using the current weight column. The weighted error and learner weight are:

$$\epsilon_t = \sum_{i=1}^N w_i \mathbf{1}(h_t(x_i) \neq y_i), \quad \alpha_t = \frac{1}{2} \log \left(\frac{1 - \epsilon_t}{\epsilon_t} \right) \quad (2)$$

Weights are then increased for misclassified rows, decreased for correctly classified rows, and normalised for the next round. Predictions, error flags, and weight updates remain as distributed DataFrame columns only scalar values such as weighted error and normalisation constants are collected to the driver. The final ensemble score is:

$$F(x) = \sum_{t=1}^T \alpha_t h_t(x) \quad (3)$$

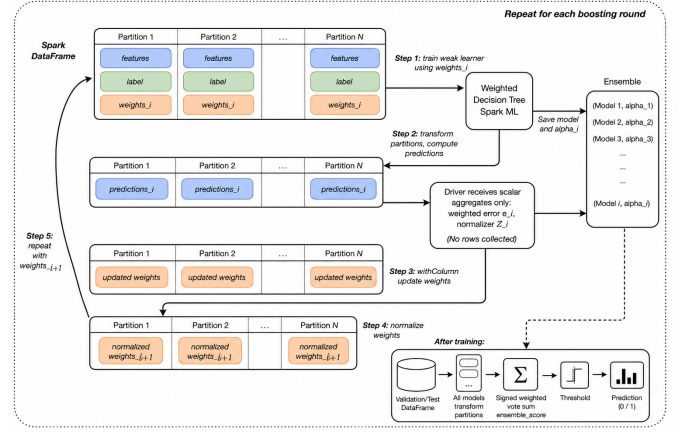


Fig. 1. Global AdaBoost Design: weak learner training, weighted error aggregation, distributed row-weight update, and ensemble voting.

TABLE I
DISTRIBUTED DATA MOVEMENT IN THE ADABOOST LOOP.

Stage	Executors	Driver
Preprocess	Feature columns and vectors	Pipeline metadata
Train tree	Weighted split statistics	Model object
Update weights	Row-level exponential update	Error and normaliser
Inference	Signed votes per partition	Metric aggregates

B. Distributed SMOTE and Custom Histogram Decision Tree

The pipeline filters to TRANSFER and CASH OUT types (zero fraud in other three types), removing 3.5M rows and improving the class ratio from 0.13% to 0.30%. Two balance discrepancy features are engineered (balanceDiffOrig, balanceDiffDest). The binary type column is one-hot encoded, yielding 8 features. Stratified 80/20 split preserves class ratio. SMOTE target: 1:4.

The binary transaction type is one-hot encoded, giving eight features. A stratified 80/20 split preserves the class ratio. SMOTE is then applied using the interpolation rule:

$$x_{new} = x_i + \lambda(x_{nn} - x_i), \quad \lambda \in [0, 1] \quad (4)$$

Three distributed SMOTE designs are compared. Broadcast SMOTE collects minority rows to the driver and broadcasts them to all workers, giving global nearest-neighbour access but communication cost $O(M \times W)$. Local SMOTE applies SMOTE within each partition using only local fraud rows, giving communication cost $O(0)$, but it can suffer from stragglers when fraud rows are unevenly distributed. Shuffle-based SMOTE repartitions only the minority class before local interpolation, improving load balance with communication cost $O(M)$.

The classifier is a custom histogram-based Decision Tree implemented with PySpark SQL rather than MLlib. Data remains cached in partitions with a `node_id` column. At each tree level, feature values are exploded, binned into equal-

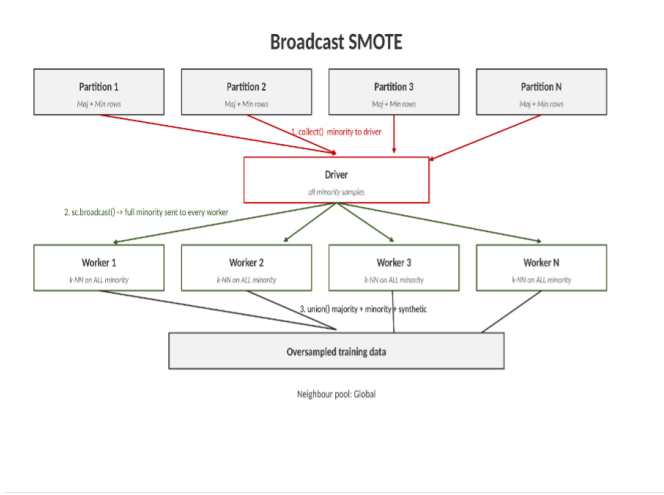


Fig. 2. Broadcast SMOTE data movement. Minority collected to driver, broadcast to all workers. Neighbour pool: global.

width bins, and aggregated by node, feature, bin, and label: $H = \text{groupBy}(\text{node}, \text{feature}, \text{bin}, \text{label}).\text{count}()$

Only the compact histogram counts are collected to the driver. The driver selects splits using Gini impurity,

$$Gini = 1 - \sum_{k=1}^K p_k^2 \quad (5)$$

then returns split decisions as PySpark column expressions. The collected summary size is bounded by $O(A \times F \times B \times K)$, where A is active nodes, F is features, B is bins, and K is classes. This reduces data movement because raw rows remain on workers and only histogram summaries are transferred. node id assignments are updated via SQL WHEN expressions added as a lazy transformation (no shuffle required). The data itself never leaves the workers only small histogram counts (KB) travel upward and split decisions return as part of Spark's normal job serialisation. The model uses $\text{max_depth}=8$, $\text{min_samples_split}=100$, and $\text{n_bins}=64$.

C. Sampling Strategies with Gradient-Boosted Trees

The third implementation evaluates local undersampling, distributed SMOTE, and global undersampling with PySpark MLlib GBTClassifier. As above, the data is filtered to TRANSFER and CASH_OUT, removing 3.59 million rows while preserving fraud-relevant records. Two balance features are engineered $\text{balance_diff} = \text{oldbalanceOrg} - \text{newbalanceOrg}$ and $\text{dest_balance_diff} = \text{newbalanceDest} - \text{oldbalanceDest}$. A stratified 80/20 split performed. Local undersampling uses $\text{sampleBy}()$ within partitions with sampling fraction:

$$\text{fraction} = \frac{N_{\text{fraud}}}{N_{\text{normal}}} = \frac{6575}{2,209,514} \approx 0.002976 \quad (6)$$

producing a 1:1 balanced set of 13,296 rows with no network transfer. Distributed SMOTE uses $\text{mapPartitions}()$, $K =$

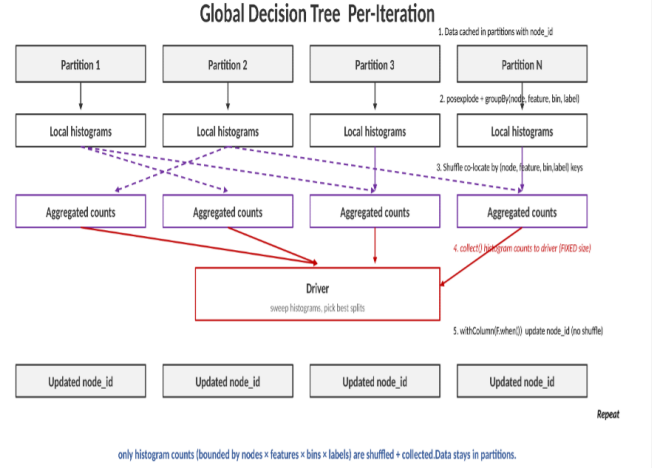


Fig. 3. Decision tree per-iteration data movement. Data stays in partitions, only histogram counts (bounded by nodes $\times$ features $\times$ bins $\times$ labels) are shuffled and collected. Split decisions return as lazy SQL transformations

3 nearest neighbours, and five synthetic rows per fraud example, producing 39,450 total fraud rows in a 2,248,964-row training set. Global undersampling computes global class counts using $\text{groupBy}().\text{count}()$ and then applies $\text{sampleBy}()$ with the globally computed fraction. It produces the same balanced row count as local undersampling but requires a shuffle. The communication cost of each strategy is:

$$\text{Comm}_{\text{local}} = O(0), \quad \text{Comm}_{\text{global}} = O(N) \quad (7)$$

The classifier uses $\text{maxIter}=10$ and $\text{maxDepth}=5$. GBT builds trees sequentially, so tree $t+1$ cannot start until tree t has completed. This makes boosting effective for difficult fraud cases but creates a distributed bottleneck because each tree requires another pass over the data and may trigger a shuffle:

$$C_{\text{GBT}} \approx O(T \times C_{\text{tree}}) \quad (8)$$

where T is the number of boosting iterations and C_{tree} is the cost of one distributed tree. Predictions are cached after generation to avoid recomputing the SMOTE lineage. SMOTE ratios of $5\times$, $50\times$, and $335\times$ are also tested to examine the effect of oversampling strength on performance and runtime.

IV. EXPERIMENTAL STUDY

The experimental study evaluates the three PySpark implementations from two perspectives, classification quality on the imbalanced fraud task and distributed behaviour under changes in data size, boosting rounds, and partition count. All the models were executed on Databricks except for the Global AdaBoost, which was run on a MacBook Pro with a 14-core Apple M4 pro processor and 48GB of RAM. The software stack included Python 3.12.12, PySpark 4.1.1 and

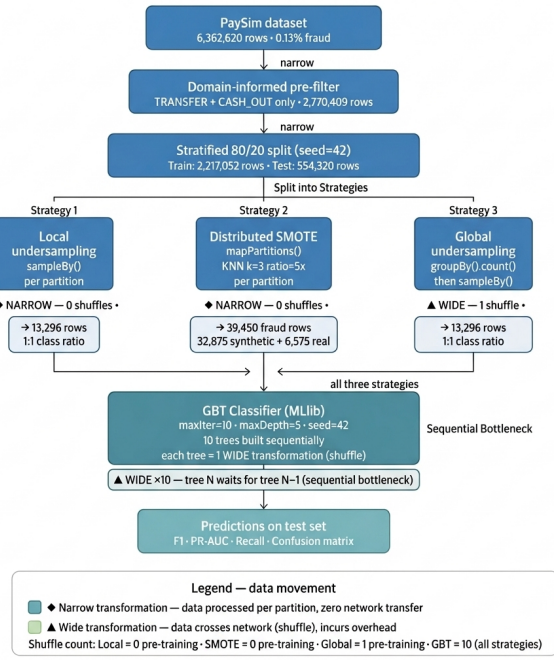


Fig. 4. GBT sampling pipeline comparing local undersampling, partition-level SMOTE, and global undersampling

TABLE II
ADABOOST VALIDATION AND TUNED-TEST CLASSIFICATION RESULTS.

Stage	Acc.	Prec.	Rec.	F1	PR-AUC
Valid, threshold	0.999954	0.9682	0.9967	0.9823	0.9889
Test, tuned	0.999947	0.9616	0.9966	0.9788	0.9877

other libraries which is included in a markdown file in the notebook at the end.

A. Class-Weighted Global AdaBoost

The AdaBoost experiments evaluate a three-round global ensemble trained with class-balanced row weights. Validation performance was first measured using the default ensemble threshold, then a validation threshold search was used to maximise fraud-class F1-score before final testing. Table II summarises the main classification results.

The three weak learners produced alpha values of 2.8562, 1.0258 and 0.7804 from weighted errors of 0.00329, 0.11389 and 0.17353. The validation threshold search selected 1.1656 as the best threshold. On the held-out test set, this produced 953,368 true negatives, 47 false positives, 4 false negatives and 1,178 true positives. These results show that class-weighted AdaBoost maintained high fraud recall while keeping false positives low after threshold tuning.

Scalability experiments measured size-up behaviour, boosting round cost and partition-count sensitivity. Runtime increased from 1.93s on 446k rows to 6.52s on 4.45M rows, which was sub-linear over the tested range. Increasing boosting rounds from one to five increased runtime from 1.93s to 9.84s, which is close to linear because each round trains one more weak learner and performs another scoring and weight

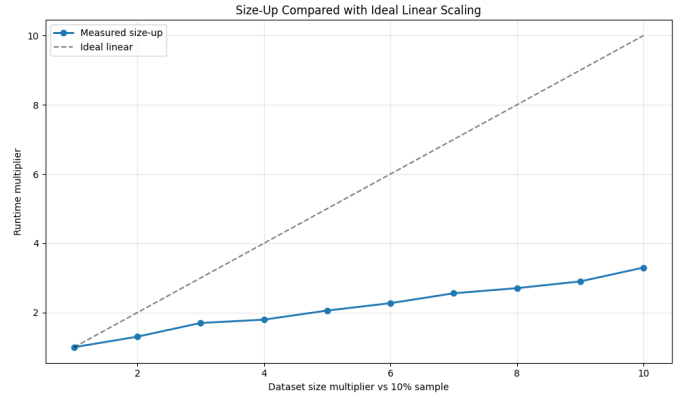


Fig. 5. AdaBoost size-up runtime compared with ideal linear scaling.

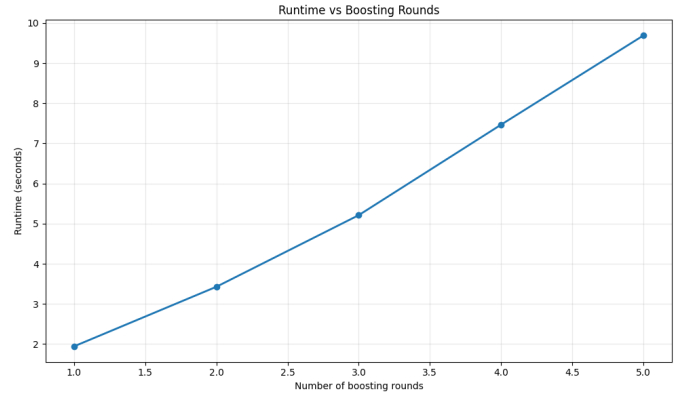


Fig. 6. AdaBoost runtime as the number of boosting rounds increases.

update pass. Partition experiments showed stable metrics, with the best runtime observed at 14 partitions.

B. Distributed SMOTE and Custom Global Decision Tree

The custom Decision Tree experiments were conducted on approximately 2.8M rows after filtering to fraud-relevant transaction types, with around 6.5k minority training samples. All SMOTE variants were evaluated on the same stratified test set. Table III compares the baseline tree without SMOTE against broadcast, local and shuffle-based SMOTE.

SMOTE improved fraud recall from 0.4048 to approximately 0.68, but precision fell from 0.9471 to about 0.117. This shows a clear precision-recall trade-off: oversampling helped the model identify more fraud cases, but it also shifted the boundary toward the minority class and produced more false positives. Classification quality was almost identical across the three SMOTE distribution strategies, so the strategy choice was mainly a runtime decision. Shuffle-based SMOTE was fastest because it moved only the minority class and improved load balance, while local SMOTE was slowest due to partition imbalance.

Partition-count experiments from 2 to 32 partitions showed that classification metrics remained effectively constant, supporting the partition-invariant behaviour of the global histogram tree. Size-up experiments showed near-constant train-

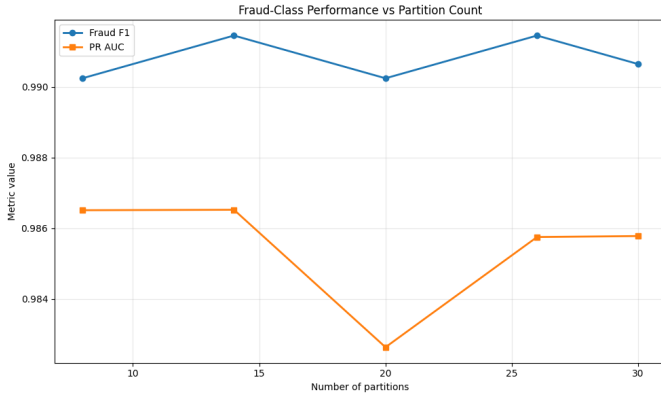


Fig. 7. AdaBoost fraud precision and recall across partition counts.

TABLE III
CUSTOM DECISION TREE CLASSIFICATION AND RUNTIME RESULTS.

Method	Acc.	Prec.	Rec.	F1	Time
No SMOTE	0.9982	0.9471	0.4048	0.5672	—
Broadcast	0.9839	0.1172	0.6801	0.2000	25.1s
Local	0.9840	0.1176	0.6795	0.2005	48.6s
Shuffle	0.9840	0.1176	0.6795	0.2005	18.1s

ing time over the tested range. This is because the histogram output collected to the driver is bounded by active nodes, features, bins and labels, rather than by the number of input rows.

C. Sampling Strategies with Gradient-Boosted Trees

The GBT experiments were run on an Azure Databricks shared student cluster. Fixed seed 42 made classification metrics reproducible but training times varied due to shared-cluster contention so timing results are interpreted by relative pattern rather than exact values.

As shown in Table IV, SMOTE 5x achieved the highest PR-AUC of 0.8244 with only 307 false alarms but missed 386 fraud cases. Local undersampling caught 99.39% of fraud but generated 17,670 false positives. Local and global undersampling produced identical results confirming the additional global shuffle gave no accuracy benefit. The main trade-off was operational SMOTE gave higher precision and fewer false alarms while undersampling gave stronger fraud recall at the cost of many more false positives. The SMOTE ratio experiment tested 5x, 50x and 335x oversampling and full balance at 335x produced the weakest F1=0.9797 due to synthetic data saturation at 335x each of 6,575 real fraud rows generates 335 synthetic neighbours using only K=3 real neighbours causing synthetic points to cluster tightly around real examples so GBT learns a constrained boundary that fails on real test fraud.

As shown in Fig. 11, partition-count experiments from 2 to 32 partitions showed stable F1 between 0.977 and 0.983 confirming partition-invariant accuracy. Local undersampling applied before repartitioning guarantees 1:1 class balance per partition regardless of count removing the main mechanism through which partitioning typically affects classification.

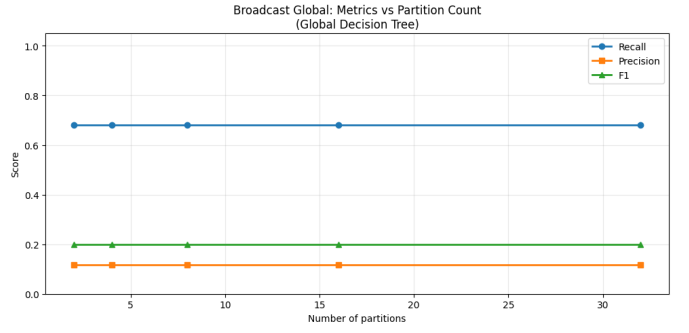


Fig. 8. Metrics constant across partition counts the global histogram is partition-invariant (RQ 3)

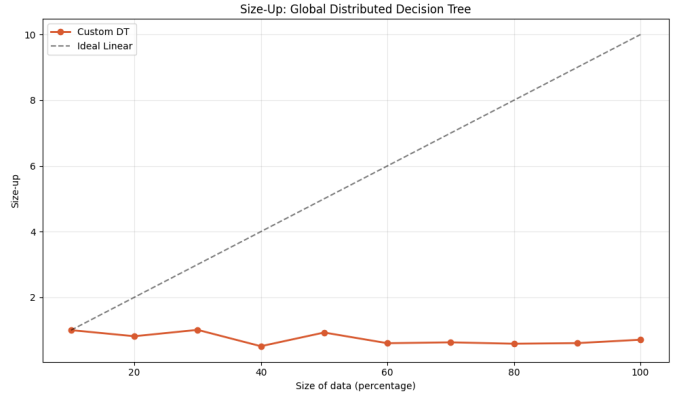


Fig. 9. Size-up: near constant training time. Histogram output bounded by nodes×features×bins×labels, not N (RQ2).

As shown in Fig. 10, Size-up experiments showed sub-linear behaviour because the balanced training set remained small (3,340-13,296 rows) at every scale so Spark scheduling overhead dominated GBT computation. F1 stayed stable between 0.978 and 0.982 across all scales confirming consistent fraud detection regardless of input size.

V. DISCUSSION

Class-weighted AdaBoost gave the strongest balanced fraud performance after threshold tuning, detecting 1,178 of 1,182 test fraud cases with only 47 false positives. This shows the value of combining class-balanced weights with validation-based threshold selection, since aggregate accuracy alone would hide the rare-class operating point.

The distributed results show that strategy choice is often a scalability decision. Broadcast, local and shuffle-based SMOTE produced similar classification quality, so runtime

TABLE IV
GBT CLASSIFICATION RESULTS. FN = FRAUD MISSED; FP = FALSE ALARMS.

Method	F1	PR-AUC	Recall	FN	FP
Local Undersample	0.9813	0.7458	0.9681	10	17,670
SMOTE 5x	0.9987	0.8244	0.9987	386	307
Global Undersample	0.9813	0.7501	0.9681	10	17,670

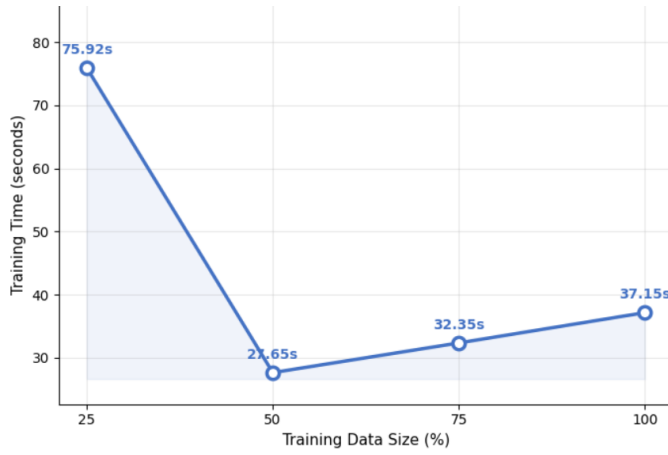


Fig. 10. GBT runtime vs training data size showing sub-linear scaling behaviour.

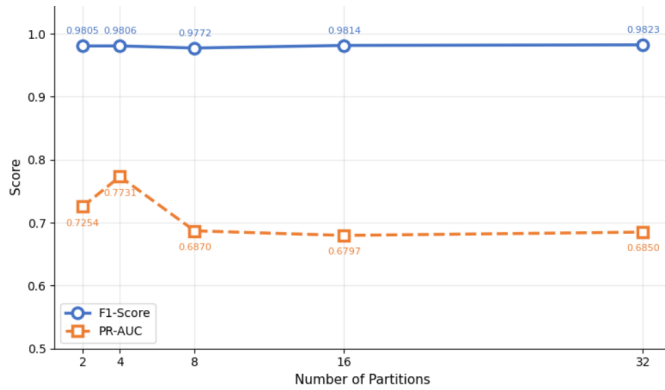


Fig. 11. GBT classification performance vs partition count confirming partition-invariant accuracy.

and data movement determined the best option. Shuffle-based SMOTE was fastest for the custom Decision Tree because only minority rows were repartitioned, while local SMOTE suffered from partition imbalance. Local and global undersampling gave identical GBT quality, meaning the global shuffle added cost without improving accuracy.

Regarding Scalability results AdaBoost scaled near-linearly with boosting rounds because each round trains another weak learner and scans the data again. The histogram Decision Tree and undersampled GBT showed sub-linear size-up over the tested range, but this does not prove asymptotic sub-linear complexity. The per row computation in decision tree (posexplode + bin assignment) completes very quickly even at full scale, so per-iteration job scheduling overhead is the dominant cost. At significantly larger scales (tens of millions of rows) the linear $O(N)$ component would begin to dominate.

A. Limitations

The main limitations are the use of a single synthetic dataset, limited repeated timing trials, shared cluster timing noise. PaySim may not fully represent real fraud behaviour, and SMOTE cannot generate fraud patterns outside the neigh-

bourhood of existing minority examples. The custom histogram Decision Tree also lacks some regularisation features available in mature MLlib models, and increasing bins or features would increase histogram communication and driver memory pressure.

VI. CONCLUSION

This report investigated distributed class-imbalance techniques for fraud detection on the PaySim dataset using PySpark. Three implementations were evaluated: class-weighted AdaBoost, distributed SMOTE with a custom histogram-based Decision Tree, and sampling-based Gradient-Boosted Trees. The results show that imbalance handling is essential because overall accuracy alone is misleading for a dataset where fraud represents only about 0.13% of transactions.

Future work should include evaluating the methods on real transaction datasets and larger clusters, repeat timing experiments under controlled resource settings, and test scales where the linear cost of row processing dominates scheduling overhead. Further improvements include cross-validated threshold tuning, hybrid sampling with class weights, approximate KNN for larger SMOTE minority sets, deeper or more numerous AdaBoost weak learners, and comparison with Random Forest or other parallel ensemble baselines.

REFERENCES

- [1] E. A. Lopez-Rojas, A. Elmir, and S. Axelsson, "Paysim: A financial mobile money simulator for fraud detection," in *Proceedings of the 28th European Modeling and Simulation Symposium (EMSS)*, Larnaca, Cyprus, 2016.
- [2] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "Smote: Synthetic minority over-sampling technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.
- [3] Y. Freund and R. E. Schapire, "A decision-theoretic generalization of on-line learning and an application to boosting," *Journal of Computer and System Sciences*, vol. 55, no. 1, pp. 119–139, 1997.
- [4] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "Lightgbm: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems*, vol. 30, 2017, pp. 3146–3154.
- [5] M. Galar, A. Fernández, E. Barrenechea, H. Bustince, and F. Herrera, "A review on ensembles for the class imbalance problem: Bagging-, boosting-, and hybrid-based approaches," *IEEE Transactions on Systems, Man, and Cybernetics, Part C: Applications and Reviews*, vol. 42, no. 4, pp. 463–484, 2012.
- [6] I. Triguero and M. Galar, *Large-Scale Data Analytics with Python and Spark*. Cambridge University Press, 2023.
- [7] M. J. Basgall, W. Hasperué, M. Naiouf, A. Fernández, and F. Herrera, "Smote-bd: An exact and scalable oversampling method for imbalanced classification in big data," *Journal of Computer Science and Technology*, vol. 18, no. 3, p. e23, 2018.
- [8] M. Juez, J. F. Diez-Pastor, C. García-Osorio, and J. J. Rodríguez, "Approx-smote: Fast smote for big data on apache spark," *Neurocomputing*, vol. 464, pp. 432–437, 2021.